

L1: Introduction; Elements of Programming

CS1101S: Programming Methodology

Boyd Anderson

16th August, 2023

- 1 What is CS1101S?
- 2 Course management
- 3 Elements of programming

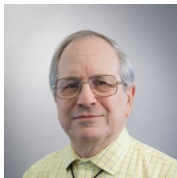
- 1 What is CS1101S?
 - Course history
- 2 Course management
- 3 Elements of programming

The goals of this course

- Understand
the *structure and interpretation of computer programs*
- Learn how to program
- Learn computational thinking
- Get glimpses into the subject areas of computer science
- Get a feeling for how computer science “ticks”
- Fall in love with computer science

Course origins

Course concept developed at MIT in the 1980s,
by Hal Abelson and Jerry Sussman



- Book: “Structure and Interpretation of Computer Programs”
- A.K.A. the “SICP”

CS1101S Legacy: The Beginning

Introduced in DISCS in Sem 1 1997/1998 as “IC1101S”

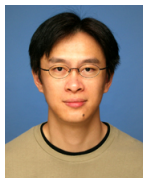
Lecturers: Jacob Katzenelson and Leong Tze Yun



Jacob renamed it to CS1101S in 1998/1999 Sem 1

CS1101S Today

Martin Henz took over the course in 2012
Adapted SICP for JavaScript instead of Scheme
Martin taught the course in 2012–2021, and he is back for 2023.
Low Kok Lim joined in 2014!



1 What is CS1101S?

2 Course management

- Weekly flow
- People
- Course overview
- Resources
- Other Matters

3 Elements of programming

1 What is CS1101S?

2 Course management

- Weekly flow
- People
- Course overview
- Resources
- Other Matters

3 Elements of programming

The weekly flow

- Wednesday 2-h **Lecture**: introduces new concepts
- Wednesday **Path**: online post-lecture quiz in Source Academy
- Thursday 1-h **Reflection**: reflects on lecture material
- Friday 1-h **Brief**: special topics
- Friday **Path**: online post-brief quiz in Source Academy
- Monday/Tuesday 2-h **Studio session**: **Master the concepts!**
- **Missions, Quests, Contests**: Show your mastery!

Homework to be submitted throughout the semester

Throughout the semester, we will have...

- 17 Paths
- 20 Missions
- 15 Quests

(some adjustment possible)

This week

- today 2-h **Lecture L1**: Elements of Programming
- today **Path P1A** on Elements of Programming
- tomorrow 1-h **Reflection R1**: choose between 10am and 2pm sessions
- Friday 1-h Mission **Brief B1**: Runology
- Friday **Path P1B** on Runology
- Friday Mission “Rune Trials” comes out, due on Tuesday, 22/8
- Monday/Tuesday 2-h **Studio session S2** on Elements of Programming

1 What is CS1101S?

2 Course management

- Weekly flow
- **People**
- Course overview
- Resources
- Other Matters

3 Elements of programming

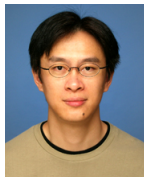
Lecturers

Their roles

Conduct **lectures** on Wednesdays and **briefs** on Fridays,
coordinate the course



Martin Henz



Low Kok Lim



Sanka Rasnayaka



Boyd Anderson

Tutors

Their role

Facilitate weekly **Reflection** sessions

Eldric Liew, Sun Bangjie, Darius Foo Tsien Wei Chow En Rong,

Wong Pin Rui Ivan,

Martin Henz, Low Kok Lim, Sanka Rasnayaka, Boyd Anderson

Source Academy Team

Their roles

Design, develop, deploy, maintain the **Source Academy**, our learning environment for CS1101S

Richard Dominick, Source Academy CTO

Chen Yanyu, Source Academy Backend Lead Developer

What is CS1101S?
Course management
Elements of programming

Weekly flow
People
Course overview
Resources
Other Matters

Staff Photos

The 2023 team



Avengers

Their role

- facilitating weekly 2-h Studio sessions,
- guiding you in the fine art of programming,
- discussing your missions and quests with you and grading them
- conducting mastery checks (more on this later)

1 What is CS1101S?

2 Course management

- Weekly flow
- People
- **Course overview**
- Resources
- Other Matters

3 Elements of programming

Course Overview

- Unit 1—Functions (textbook Chapter 1)
 - Getting acquainted with the elements of programming, using **functional abstraction**
 - Learning to read programs, and using the **substitution model**
 - Example applications: Runes, curves
- Unit 2—Data (textbook Chapter 2)
 - Getting familiar with data: pairs, lists, trees
 - Searching in lists and trees, sorting of lists
 - Example application: sound processing

Course Overview, continued

- Unit 3—State (parts of textbook Chapter 3)
 - Programming with stateful abstractions
 - Arrays, loops, searching in and sorting of arrays
 - Reading programs using the environment model
 - Example applications: robotics, video processing
- Unit 4—Beyond (parts of textbook Chapters 3 and 4)
 - Streams
 - Metalinguistic abstraction
 - Computing with Register Machines / Logic Programming / Concurrency

Special Event: Sumobot and Robot Talent Show



Saturday, October 21st, 2023.
Details to come.

Assessment timeline

- **Reading Assessment RA1** on Friday Week 4: 8/9 (Unit 1)
- **Recess Week**
- **Midterm** Assessment on Wednesday Week 7: 4/10 (Unit 2)
- **Reading Assessment RA2** on Friday Week 10: 27/10 (Unit 3)
- **Practical Assessment** on Saturday Week 11: 4/11 (Unit 4)
- **Final** Assessment: 27/11

Assessment components

- 14%: Source Academy XP (Missions, Quests, Paths, Studios, ...): 24,000 XP
- 5%: Studio: attendance 3%, effort 2%
- 5%: Reflection attendance
- 5%: Reading Assessment 1, Week 4
- 3%: Mastery Check 1, any time
- 15%: Midterm Assessment, Week 7
- 8%: Reading Assessment 2, Week 10
- 12%: Practical Assessment, Week 11
- 3%: Mastery Check 2, any time
- 30%: Final Assessment

14%: based on XP in Source Academy

Your homework

Homework (Paths, Missions, Quests, Studios) earn XP.

Early submission bonus

Submit Missions and Quests within 3 days of release: 75 XP bonus. Submit Paths within 1 day of release: 25 XP bonus.

Contests

Look out for announcements. Winners & runners-up receive XP. You will also earn XP for participating in the contest voting.

Miscellaneous special topics

Extra XP to deserving individuals; look out for announcements.

1 What is CS1101S?

2 Course management

- Weekly flow
- People
- Course overview
- **Resources**
- Other Matters

3 Elements of programming

Resources

Textbook: SICP JS

Text Book can be found here: [SICP JS](#) A paperback veriso can be bought [here](#)

Weblinks: Source, Source Academy, Ed Discussion, calendars
Canvas > CS1101S > [Important and Useful Weblinks](#)

Course FAQs can be found here

Canvas > CS1101S > [Frequently Asked Questions](#)

Need help?

- Ed Discussion forums:
<https://edstem.org/us/courses/42391/discussion/>
Many of you are signed up, already. If you cannot sign up, please email Martin
- PM your Avenger (WhatsApp/FB/Telegram/you-name-it: check what channel they prefer)
- **Forum Hours.** Every Tuesday 6:30pm-8:30pm, starting from Week 3. Venue is COM1 basement.

Need help?

- Email your Reflection instructor:
Canvas > CS1101S > **Teaching Staff**
- Email lecturers
 - Martin: henz@comp.nus.edu.sg
 - Kok Lim: lowkl@comp.nus.edu.sg
 - Sanka: sanka@nus.edu.sg
 - Boyd: boyd@comp.nus.edu.sg

Academic Honesty

The University takes a strict view of cheating in any form. Any student who is found to have engaged in such misconduct will be subjected to disciplinary action by the University.

Plagiarism

The practice of taking someone else's work or ideas and passing them off as one's own (e.g. copying from your classmates, seniors, books and/or online resources).

Be sure read the [Code of Student Conduct](#)

AI and You

- You have probably already used them for fun or for your work. The teaching team is definitely using them to improve the course.
- These AI tools will be a great resource for you in your learning journey. Make sure you try them out. Ask your Avenger how they are using them.
- Good news: These AI tools know our material very well. There might still be mistakes in their answers, so make sure you verify the answers by asking your Avenger, tutor, or lecturer.

AI and You

- There will be plenty of time to get acquainted with these tools in later courses.
- For this course, we need you to use your own understanding to solve our assignments. This practice is important.
- You won't be able to use AI tools in assessments.
- If we use them to just get an answer, we are cheating ourselves out of a learning opportunity.

1 What is CS1101S?

2 Course management

3 Elements of programming

- Processes and programming
- Primitive expressions, 1.1.1
- Operator combinations, 1.1.1
- Evaluating combinations, 1.1.3
- Naming abstraction, 1.1.2
- Functional abstraction, 1.1.4
- Predicates and conditional expressions, 1.1.6

1 What is CS1101S?

2 Course management

3 Elements of programming

- Processes and programming
- Primitive expressions, 1.1.1
- Operator combinations, 1.1.1
- Evaluating combinations, 1.1.3
- Naming abstraction, 1.1.2
- Functional abstraction, 1.1.4
- Predicates and conditional expressions, 1.1.6

Processes

Definition (from Wikipedia)

A process is a set of activities that interact to produce a result. The activities unfolds according to patterns that *describe* or *prescribe* the process.

Examples

Processes are everywhere. They permeate our nature and culture:

- Galaxies and solar systems follow *formation* processes.
- Metabolic pathways in our bodies are *biological* processes.
- Political parties, legislature, courts, etc. follow processes.
- Industrial production is governed by processes.

Computational processes

Definition

A *computational process* is a set of activities in a computer, designed to achieve a desired result.

Our task

Here we are concerned about how this *design* happens.

Our design method

We use *programs* to prescribe how the computational process unfolds.

What is programming?

People in focus

As the complexity of computer systems increases, *communication* between architects, designers, programmers, operators and users becomes increasingly important.

Programs as communication devices

Since programs prescribe the computational processes at the heart of these systems, they can play a central role in the *human* communication during their construction and operation.

Our motto

Programming is communicating computational processes.

Our programming environment: Source Academy

Website designed for CS1101S

Has just what you need for understanding the *structure and interpretation of computer programs*.

Research

Source Academy is also an *educational research tool*: We want to study what happens when people like you learn how to program.

Your first login to Source Academy

Consent

You can be part of our research by letting us use your programs anonymously. But if you don't want to: No worries: There will be **no penalty** if you don't consent.

Our ship

You think you are studying at NUS? Wrong! You are cadets in... the Source Academy!

Visit <https://sourceacademy.nus.edu.sg>, decide on consent and click on "Playground". (Source Academy opens after the lecture.)

1 What is CS1101S?

2 Course management

3 Elements of programming

- Processes and programming
- Primitive expressions, 1.1.1
- Operator combinations, 1.1.1
- Evaluating combinations, 1.1.3
- Naming abstraction, 1.1.2
- Functional abstraction, 1.1.4
- Predicates and conditional expressions, 1.1.6

Elements of programming

Every powerful language provides...

- Primitives
- Combination
- Means of abstraction

Primitive expressions

- Numerals: 0, -42, 486
- Numerals use decimal notation
- Our interpreter can evaluate numerals, resulting in the numbers they represent

A detail

Expressions are not programs in Source

Instead they can be turned into programs using a semicolon.

Example

486; is a program.

1 What is CS1101S?

2 Course management

3 Elements of programming

- Processes and programming
- Primitive expressions, 1.1.1
- **Operator combinations, 1.1.1**
- Evaluating combinations, 1.1.3
- Naming abstraction, 1.1.2
- Functional abstraction, 1.1.4
- Predicates and conditional expressions, 1.1.6

Means of Combination: Operators

Examples

5 * 99;

25 - (4 + 2) * 3;

Notation as usual

operator between operands: *infix notation* with *precedences*

Evaluating Operator Combinations

But exactly how...

...do we evaluate

$(2 + 4 * 6) * (3 + 12)$

???

Evaluation of expressions

Demonstration: Evaluation of expressions

Evaluation of expressions

1 + 2 * 3 + 4

Replace “eligible” subexpression with result

(1 + (2 * 3)) + 4

(1 + 6) + 4

7 + 4

11

Story: <https://sourceacademy.nus.edu.sg/stories/view/1>

1 What is CS1101S?

2 Course management

3 Elements of programming

- Processes and programming
- Primitive expressions, 1.1.1
- Operator combinations, 1.1.1
- Evaluating combinations, 1.1.3
- **Naming abstraction, 1.1.2**
- Functional abstraction, 1.1.4
- Predicates and conditional expressions, 1.1.6

Means of Abstraction: Naming

Example

```
const size = 2;  
5 * size;
```

Evaluation of constant declarations

```
const a = 3 * 5;  
a + 5;
```

Evaluate what is after the =

```
const size = 3;  
2 * size;
```

Then replace that name in rest of the program with that value.

```
2 * 3;  
  
6;
```

Story: <https://sourceacademy.nus.edu.sg/stories/view/2>

Pre-declared names

Pre-declared constants

Source has a few names pre-declared. For example, the name `math_PI` refers to the constant π .

Pre-declared functions

In addition to operators such as `+`, Source has pre-declared functions. For example, `math_sqrt` is the square root function.

Function application expressions

Functions can be applied as usual in mathematics, for example

```
math_sqrt(15);
```

applies the pre-declared square root function to 15.

1 What is CS1101S?

2 Course management

3 Elements of programming

- Processes and programming
- Primitive expressions, 1.1.1
- Operator combinations, 1.1.1
- Evaluating combinations, 1.1.3
- Naming abstraction, 1.1.2
- **Functional abstraction, 1.1.4**
- Predicates and conditional expressions, 1.1.6

Means of Abstraction: Compound functions

Example

```
function square(x) {  
    return x * x;  
}
```

What does this statement mean?

Like constant declarations, this *function declaration* declares a name, here `square`. The value associated with the name is a function that takes an argument `x` and produces (returns) the result of multiplying `x` by itself.

Review: Function application

Recall from the `math_sqrt` example

We apply a function by supplying its arguments in parentheses.

Example

```
function square(x) {  
    return x * x;  
}  
square(14);
```

More examples

```
square(21);
```

```
square(2 + 5);
```

```
square(square(3));
```


1 What is CS1101S?

2 Course management

3 Elements of programming

- Processes and programming
- Primitive expressions, 1.1.1
- Operator combinations, 1.1.1
- Evaluating combinations, 1.1.3
- Naming abstraction, 1.1.2
- Functional abstraction, 1.1.4
- **Predicates and conditional expressions, 1.1.6**

Boolean values

Boolean values

The two boolean values `true` and `false` represent *answers* to yes-no questions

Predicates

A *predicate* is a function or an expression that returns or evaluates to a boolean value.

Examples

```
true;  
false;  
x >= 1;
```

Another predicate

Example

```
function adult(x) {  
    return x >= 18;  
}
```

Conditional expressions

Why?

We would like to check something, e.g. answer a yes/no question, and return a different value, depending on the answer

Back to the example

```
adult(my_age) ? enter_club() : go_to_movie()
```

Example: abs function

To compute the absolute value of a number, check if it is greater or equal 0. If yes, return the number unchanged, and if no, return the number negated.

May the Source be with you!

The Source Academy is open!

Look out for your first Path “Elements of Programming”.

Welcome on board!